# Universidade Presbiteriana Mackenzie

Faculdade de Computação e Informática - FCI

# Projeto Prático 1 (LAB1)
# Analisador de Logs de Sensores IoT com Pthreads

Computação Paralela

Prof. Dr. Jean M. Laine

## 1. Introdução: O Desafio da Internet das Coisas (IoT)

A Internet das Coisas (IoT) está revolucionando a forma como monitoramos e interagimos com o mundo. Cidades inteligentes, indústrias 4.0, data centers e hospitais modernos possuem milhares de sensores gerando dados continuamente: temperatura, umidade, pressão, consumo de energia, batimentos cardíacos, entre outros.
Um data center, por exemplo, pode ter mais de 10.000 sensores de temperatura e umidade enviando leituras a cada segundo. Isso gera **864 milhões de registros por dia**! Analisar esses dados em tempo real é crucial para:

- Detectar superaquecimento antes que cause falhas
- Identificar sensores com comportamento anômalo
- Otimizar o consumo de energia do sistema de refrigeração

Neste projeto, você irá desenvolver um analisador paralelo de logs de sensores, capaz de processar milhões de leituras e extrair estatísticas relevantes em segundos, utilizando o poder das threads.

## 2. Objetivos do Projeto

Ao final deste projeto, você será capaz de:

- Desenvolver uma aplicação em C para processamento de dados em larga escala
- Implementar soluções sequencial e paralela com Pthreads
- Utilizar mutex para proteção de seções críticas
- Calcular médias, variâncias e detectar anomalias em dados de sensores
- Analisar escalabilidade com Speedup, Eficiência e Lei de Amdahl

## 3. Descrição do Problema: Monitoramento de Data Center

Você deve criar um programa em C que analise um arquivo de log contendo leituras de sensores de um data center. Cada linha representa uma leitura de um sensor em um determinado instante.

### Formato do Log

```
sensor_001 2026-03-19 08:00:00 temperatura 23.5 umidade 55.2 status OK
sensor_002 2026-03-19 08:00:00 temperatura 24.1 umidade 52.8 status ALERTA
sensor_003 2026-03-19 08:00:00 energia 450.5 corrente 3.8 status OK
sensor_001 2026-03-19 08:00:01 temperatura 23.6 umidade 55.1 status OK
sensor_004 2026-03-19 08:00:01 temperatura 78.3 umidade 10.2 status CRITICO
```

### Campos da linha

| Campo | Descrição |
|---|---|
| sensor_XXX | Identificador único do sensor (ex: sensor_001 até sensor_999) |
| Data e hora | Formato AAAA-MM-DD HH:MM:SS |
| tipo | Tipo da leitura: temperatura, umidade, energia, corrente, pressão |
| valor | Valor numérico da leitura (float) |
| status | OK, ALERTA ou CRITICO |

### Estatísticas a serem extraídas

Seu programa deve calcular:

1. **Média de temperatura por sensor:** Para cada sensor de temperatura, a média de todas as suas leituras.
2. **Sensor mais instável:** O sensor de temperatura com o maior desvio padrão (variação térmica).
3. **Total de alertas:** Quantas linhas possuem status ALERTA ou CRITICO.
4. **Consumo total de energia:** Soma de todas as leituras do tipo "energia".

**Desafio: Detecção de Anomalias**

Implemente um detector de anomalias: uma leitura é considerada anômala se seu valor se desviar mais de **3 desvios padrão** da média histórica daquele sensor.
**Exemplo:** Se a temperatura média do sensor_001 é 23.5 com desvio padrão 0.5, leituras abaixo de 22.0 ou acima de 25.0 são anômalas.
Liste todas as anomalias encontradas com data/hora e valor.

## 4. Etapas do Projeto

### Etapa 1: Geração dos Dados

1. **Script gerador:** Utilize o script `generate_sensor_log.py` (disponível no Moodle) para gerar o arquivo de log.

```
python generate_sensor_log.py --sensores 100 --dias 7 --output sensores.log
```

Listing 1: Exemplo de uso do gerador

2. O script permite configurar:
   - Número de sensores (ex: 50, 100, 500)
   - Período em dias (ex: 1 dia = 86.400 leituras por sensor a 1Hz)
   - Injetar anomalias artificiais para testar seu detector
3. **Arquivo de teste:** Gere um arquivo com **pelo menos 10 milhões de linhas** para que a diferença de desempenho entre versões seja significativa.

### Etapa 2: Versão Sequencial

Crie o programa `sensor_analyzer_seq.c` que:

1. **Escolha e implemente a estrutura de dados mais adequada** para armazenar as linhas do arquivo em memória. Justifique sua escolha no relatório considerando:
   - Eficiência de acesso para processamento sequencial
   - Consumo de memória vs. desempenho
   - Facilidade de implementação do parsing
2. **Projete as estruturas de dados** para armazenar as estatísticas:
   - Médias por sensor (com soma e contagem para cálculo incremental)
   - Desvio padrão (necessário armazenar soma dos quadrados)
   - Totais acumulados (energia, alertas)
3. Para cada linha, faça o parsing e atualize as estruturas de dados
4. Ao final, exiba:
   - Média de temperatura por sensor (apenas os 10 primeiros para não poluir a tela)
   - Sensor mais instável (nome e desvio padrão)
   - Total de alertas
   - Consumo total de energia
   - Tempo de execução

## Etapa 3: Versão Paralela com Mutex

Crie o programa `sensor_analyzer_par.c` seguindo a abordagem de estado compartilhado:

1. Receber o número de threads como argumento: `./sensor_analyzer_par 4 sensores.log`
2. **Escolha e implemente a estrutura de dados mais adequada** para armazenar as linhas do arquivo em memória. Justifique sua escolha no relatório considerando:
   - Eficiência de acesso para divisão entre threads
   - Consumo de memória vs. desempenho
   - Facilidade de particionamento do trabalho
3. Dividir o trabalho entre as threads de forma balanceada (ex: blocos contíguos de linhas, divisão por bytes, etc.)
4. Estruturas globais compartilhadas com mutex para:
   - Médias por sensor (atualizadas incrementalmente)
   - Somatório de energia
   - Contador de alertas
5. Cada thread processa sua fatia e, para cada leitura relevante:
   (a) Trava o mutex
   (b) Atualiza a estrutura global
   (c) Destrava o mutex
6. Thread principal aguarda todas com `pthread_join`
7. Ao final, exiba:
   - Média de temperatura por sensor (apenas os 10 primeiros para não poluir a tela)
   - Sensor mais instável (nome e desvio padrão)
   - Total de alertas
   - Consumo total de energia
   - Tempo de execução

## Etapa 4: Desafio - Otimização de Desempenho

A versão paralela com mutex global é correta, mas pode sofrer com contenção (contention) devido ao travamento constante da seção crítica.
**Seu desafio**: Implemente uma versão otimizada do analisador paralelo que minimize o overhead de sincronização e obtenha melhor escalabilidade.

1. **Objetivo:** Alcançar o maior speedup possível para 8 threads.
2. **Liberdade de implementação:** Você pode modificar estruturas de dados, estratégias de sincronização ou a divisão do trabalho.
3. **O que entregar:** Um programa `sensor_analyzer_optimized.c` (nome sugestivo) e uma seção no relatório comparando:
   - Qual estratégia você utilizou e por quê
   - Gráfico comparativo entre versão simples e otimizada
   - Análise do ganho de desempenho obtido

**Dica**

Pense em como reduzir a frequência de travamento do mutex ou até mesmo eliminá-lo em partes do código. Acumuladores locais? Múltiplos mutexes? Particionamento por sensor?

---

## Etapa 5: Análise de Escalabilidade

1. Meça o tempo da versão sequencial ($T_{seq}$)
2. Para as versões paralelas, meça com 1, 2, 4, 8 e 16 threads
3. Calcule:
   - Speedup: $S(p) = T_{seq}/T_{par}(p)$
   - Eficiência: $E(p) = S(p)/p$
4. Gere um gráfico de Speedup vs. Número de Threads

## 5. Formato da Entrega

Arquivo `.zip` contendo:

1. **Código-fonte:**
   - `sensor_analyzer_seq.c`
   - `sensor_analyzer_par.c` (versão com mutex global)
   - `sensor_analyzer_optimized.c` (versão otimizada - desafio)
   - `Makefile`
   - `generate_sensor_log.py` (opcional, se modificaram)
2. **Relatório PDF (`relatorio.pdf`)** contendo as seguintes seções:

   **1. Introdução**

   Contextualização do problema e visão geral da solução proposta. Apresente os objetivos do trabalho e a organização do relatório.

   **2. Etapa 1: Geração dos Dados**

   - Parâmetros utilizados na geração do arquivo de log (número de sensores, dias, total de linhas, probabilidade de anomalias).
   - Características do arquivo gerado (tamanho em MB, distribuição de sensores por tipo, linha de comando utilizada).
   - Inclua um print do início do arquivo (primeiras 10 linhas) para verificação do formato.

   **3. Etapa 2: Versão Sequencial**

   - **Estruturas de dados para as linhas:**
     - Apresente as structs criadas (com trecho de código).
     - Por que essa estrutura foi escolhida? Compare com pelo menos UMA alternativa rejeitada (ex: "Considerei usar lista encadeada, mas optei por array porque...").
     - Calcule o consumo de memória total estimado para armazenar o arquivo em memória.
   - **Estruturas de dados para as estatísticas:**
     - Apresente as structs criadas para armazenar médias, desvios e totais.
     - Como você implementou o cálculo do desvio padrão sem armazenar todas as leituras? (fórmula incremental)
     - Justifique a escolha da estrutura de busca/localização de sensores (array indexado por ID? hash? lista?).
   - **Implementação:**
     - Explique o algoritmo de parsing. Como você extrai os campos de cada linha? (uso de `strtok`, `sscanf`, parsing manual?)

  - Inclua um fluxograma ou pseudocódigo do processamento principal.
  - Destaque trechos do código que julgou mais importantes ou desafiadores.
- **Resultados:**
  - Tempo de execução da versão sequencial ($T_{seq}$): _____ segundos.
  - Sensor mais instável encontrado: ID _____ com desvio padrão _____.
  - Total de alertas (ALERTA + CRITICO): _____.
  - Consumo total de energia: _____ Wh.
  - *(Cole o print da execução completa da versão sequencial, com TODAS as saídas visíveis)*

4. **Etapa 3: Versão Paralela com Mutex**

- **Estruturas de dados:**
  - Adaptações necessárias para o paralelismo? As mesmas estruturas da versão sequencial foram reutilizadas?
  - Quais variáveis foram compartilhadas e quais foram mantidas como locais? Justifique.
  - Apresente as structs globais que necessitaram de proteção com mutex.
- **Divisão do trabalho:**
  - Como as linhas foram distribuídas entre as threads? (blocos contíguos, intercalado, divisão por bytes)
  - Qual o tamanho de cada bloco para 4 threads? E para 8? Apresente os cálculos.
  - Houve tratamento para sobras/desbalanceamento? Como?
- **Sincronização:**
  - Em quais pontos da thread o mutex é travado?
  - Apresente trecho de código mostrando a seção crítica protegida.
  - Há risco de deadlock? Como você garantiu que não ocorre?
- **Resultados:**
  - Tabela com tempos e speedups para 1, 2, 4, 8 e 16 threads:

| Threads | Tempo (s) | Speedup |
|---|---|---|
| 1 | | 1,00 |
| 2 | | |
| 4 | | |
| 8 | | |
| 16 | | |

  - *(Cole os prints da execução para cada configuração)*

5. **Etapa 4: Desafio - Versão Otimizada**

- **Estratégia de otimização:**
  - Descreva DETALHADAMENTE a abordagem utilizada.

  - Por que essa estratégia deveria apresentar melhor desempenho que a versão com mutex global?
  - Que trade-offs você considerou? (ex: memória vs. velocidade, complexidade vs. ganho)
- **Implementação:**
  - Apresente trechos de código que diferem da versão anterior.
  - Como é feita a combinação dos resultados parciais ao final?
  - Houve necessidade de novas estruturas de dados? Quais?
- **Resultados:**
  - Tabela com tempos e speedups para 1, 2, 4, 8 e 16 threads:

| Threads | Tempo (s) | Speedup |
|---|---|---|
| 1 | | 1,00 |
| 2 | | |
| 4 | | |
| 8 | | |
| 16 | | |

  - *(Cole os prints da execução para cada configuração)*

**6. Análise Comparativa e Discussão**

- **Gráfico comparativo:** Speedup vs. Número de Threads para ambas as versões paralelas.

(Incluir gráfico gerado em Excel, Python ou ferramenta similar)

- **Análise de escalabilidade:**
  - Calcule a eficiência para cada configuração: $E(p) = S(p)/p$.
  - A partir de quantas threads a eficiência cai abaixo de 0,7? O que isso indica?
  - Aplique a Lei de Amdahl: $S(p) = \frac{1}{(1-P)+\frac{P}{p}}$. Qual fração $P$ do código é efetivamente paralelizável? Mostre os cálculos.
- **Comparação entre as versões:**
  - Em qual número de threads a versão otimizada começou a superar a versão mutex? Ou não superou?
  - Se superou, qual o ganho percentual máximo obtido? Ex: "Para 8 threads, a versão otimizada foi 30% mais rápida".
  - Se NÃO superou, por que a estratégia escolhida não funcionou como esperado? O que você faria diferente?
- **Detector de anomalias (se implementado):**
  - Quantas anomalias foram detectadas?
  - Liste as 5 primeiras anomalias encontradas (data/hora, sensor, valor).
  - Confira se algumas delas correspondem às anomalias artificiais injetadas pelo gerador. Qual a taxa de acerto?

- **Gargalos identificados:**
  - Onde está o principal limitador de desempenho: I/O, sincronização, desbalanceamento, overhead de criação de threads?
  - Utilize ferramentas como `perf` ou `time` para justificar sua resposta.
  - Que sugestões você daria para melhorar ainda mais o desempenho?

**7. Conclusão**

- Síntese dos principais aprendizados com o projeto.
- Principais dificuldades encontradas e como foram superadas.
- O que você faria diferente se fosse implementar novamente?
- Contribuição do trabalho para sua formação em programação paralela.

**Apêndice: Prints de Execução Completos**

- Print da geração do arquivo (comandos utilizados)
- Print da versão sequencial
- Prints da versão mutex para 1, 2, 4, 8, 16 threads
- Prints da versão otimizada para 1, 2, 4, 8, 16 threads
- Todos os prints devem conter a data/hora do sistema visível ou identificação do grupo

**Observação:** Relatórios com respostas genéricas, sem evidências concretas do código desenvolvido ou sem os prints de execução serão desconsiderados.

## 6. Critérios de Avaliação

| Critério | Descrição | Peso |
|---|---|---|
| **Corretude** | Versões sequencial e paralela produzem os mesmos resultados | 1,5 |
| **Estruturas de Dados** | Escolha adequada e justificada das estruturas | 1,0 |
| **Implementação Paralela** | Uso correto de Pthreads e mutex (divisão, join, proteção) | 2,5 |
| **Análise de Desempenho** | Tabelas, gráficos, prints e cálculos de speedup | 1,5 |
| **Desafio (Otimização)** | Estratégia criativa e melhoria real de desempenho | 1,0 |
| **Relatório** | Clareza, profundidade da análise e conclusão | 2,5 |

**Nota máxima:** 10,0

## 7. Regulamento de Apresentação e Defesa

### Apresentação Obrigatória

1. A apresentação é obrigatória para todos os integrantes
2. Ausência = nota zero para o aluno ausente

### Defesa do Código

1. Cada aluno deve demonstrar domínio do código
2. O professor pode pedir alterações durante a apresentação
3. Desconhecimento do código = nota zero

### Originalidade

1. Códigos copiados (de colegas ou IA) = nota zero para todos os envolvidos
2. É permitido consultar referências, mas o código deve ser de autoria do grupo

### Entrega

1. Única via: Moodle até a data limite
2. Atrasos = não aceitos

## 8. Organização dos Grupos

- Grupos de **2 ou 3 alunos**. Não é permitido trabalhos individuais.
- Exceções devem ser comunicadas ao professor com antecedência